RESTAURANT WORKFORCE OPERATIONS

TableTime Staff

Project summary and remaining work

A clear record of what was built, what the app can do, the technology and services used, what has been verified, and the exact path from today's working release candidate to public store launch.

98% OVERALL	**100%** DESIGN / MVP	**100%** DATABASE
100% LIVE BACKEND	**92%** ADVANCED	**90%** STORE

CURRENT OUTCOME

A working cross-platform restaurant scheduling, attendance, requests, and staff-management platform is live on the web, connected to a secured Supabase backend, and packaged as verified Android release artifacts.

Public launch is not yet complete. Hosted Auth redirect configuration, selected real-user E2E checks, physical-device validation, iOS signing/build, truthful owner metadata, and store submissions still require completion.

PROJECT OVERVIEW

What we built together

TableTime started as a restaurant clock-in, clock-out, and scheduling idea. It is now a multi-role workforce product with a live backend, tenant isolation, Android artifacts, release automation, health checks, and a deployed web application.

PHASE	DELIVERED	STATE
1. Product foundation	Responsive manager and employee layouts, dashboard, time clock, weekly schedule, team directory, request center, and versioned demo persistence.	VERIFIED
2. Secure backend	Supabase Auth and PostgreSQL, 10 RLS-protected tables, role isolation, secure RPCs, indexes, migrations, and generated TypeScript contracts.	VERIFIED
3. Live workflows	Live roster, schedule publishing, attendance, breaks, requests, invitations, recovery, account deletion, and Realtime workspace refresh.	BUILT + TESTED
4. Reliability	Bounded retry, foreground resync, visible sync health, app error boundary, CI quality gate, and six-hour production endpoint checks.	VERIFIED
5. Native release	Branded icon and splash, signed Android preview APK, Android 15 emulator validation, and signed Play Store AAB versionCode 3.	ANDROID READY
6. Store preparation	Listing copy, legal/data-safety drafts, review notes, Play icon/feature graphic, and ten accepted-dimension screenshot drafts.	IN PROGRESS

Current release state

The production web app is live. The replacement Android APK passed an Android 15 emulator launch with no runtime/React Native errors, and the replacement AAB is signed, structurally valid, and ready for a Play test-track upload.

Honest scope boundary

Approximately 98% describes engineering progress, not store approval. TableTime cannot be called 100% launched until the account-bound configuration, remaining real-user tests, iOS release, signed-native screenshots, and Apple/Google submissions are complete.

PRODUCT CAPABILITIES

What the app can do

The interface and data access adapt to the authenticated database role. Owners and managers operate the restaurant; employees see only the schedules, punches, requests, and actions permitted to their own account.

CAPABILITY	OWNER / MANAGER	EMPLOYEE	STATE
Accounts and roles	Create a restaurant, manage staff, invite workers, and use manager navigation.	Invitation sign-in, password setup/recovery, and employee-only navigation.	BUILT
Dashboard	Active workers, labor hours, upcoming shifts, request badges, and floor status.	Personal work status, next shift, and quick time-clock actions.	LIVE
Team management	Add/edit employee details, role, location, hourly rate, phone, email, and active status.	Read only the employee's protected personal roster row.	BUILT
Scheduling	Create timezone-aware draft shifts and publish weekly schedules.	View only personal published shifts.	LIVE
Time clock	View attendance and recorded hours.	Clock in/out, start/end breaks, and review punch history.	LIVE
Requests	Approve or decline time-off requests with reviewer attribution and audit events.	Submit date-ranged time off and follow request status.	BUILT
Account deletion	Sole-owner safeguard and history-preserving de-identification.	Password reauthentication plus explicit DELETE confirmation.	BUILT
Live sync	Automatic RLS-scoped refresh, retry, and visible connection state.	Personal updates refresh across clients and foreground resume.	VERIFIED

Cross-platform delivery

ANDROID	iOS	WEB
Signed preview APK verified on Android 15. Signed AAB versionCode 3 is ready for Play testing.	Expo config and bundle ID are ready. Apple credentials, IPA build, TestFlight, and native QA remain.	Responsive production SPA is deployed on Vercel with live Supabase data and public legal routes.

ARCHITECTURE

How the system works

One Expo codebase delivers Android, iOS, and web. Supabase owns identity and server-authoritative workforce data; GitHub, Vercel, and EAS provide source control, automated checks, web hosting, signing, and native builds.

CLIENTS	SECURE BACKEND	DELIVERY
Android APK/AAB iOS-ready Expo app Responsive web SPA	Supabase Auth + PostgreSQL RLS + narrow RPCs Edge Functions + Realtime	GitHub + Actions Vercel web hosting Expo Application Services

LAYER	IMPLEMENTATION	WHY IT MATTERS
Client	Expo SDK 57, React Native 0.86, React 19, TypeScript, and React Native Web.	One maintained product experience across phone, tablet, and browser.
Identity	Supabase Auth sessions, invitation/recovery links, and roles read from protected memberships.	Users cannot promote themselves with editable client metadata.
Data	PostgreSQL with 10 application tables, tenant keys, indexes, RLS, and 14 ordered migrations.	Restaurant records stay isolated even if a client is modified.
Writes	Narrow RPCs for owner setup, attendance, breaks, requests, invitations, and deletion.	Authorization decisions and timestamps are enforced on the server.
Realtime	RLS-protected Postgres Changes, debounce, retry, and foreground resync.	Workspace updates arrive without exposing other restaurants' data.
Operations	CSP/HSTS headers, error boundary, CI gate, and scheduled health checks.	Release defects and public outages become visible and recoverable.

Database model Organizations, profiles, memberships, locations, employees, shifts, time entries, break entries, staff requests, and audit events. Composite foreign keys and RLS enforce tenant boundaries.

TECHNOLOGY INVENTORY

Websites, services, and libraries used

Only technologies evidenced in the repository or verified project history are included. Test passwords, signing keys, tokens, and other private credentials are intentionally excluded.

SERVICE	USE IN TABLETIME	STATE
GitHub	Source repository, history, release-quality CI, and production-health checks.	CONNECTED
Vercel	Production web hosting, HTTPS, SPA routes, and security headers.	LIVE
Supabase	Auth, PostgreSQL, RLS, RPCs, Realtime, and Edge Functions.	LIVE; AUTH URL PENDING
Expo / EAS	Cross-platform framework, project environments, Android signing, and cloud builds.	ANDROID BUILT
Google Play Console	Internal/closed testing and production distribution.	ACCOUNT/APP PENDING
Apple Developer / App Store Connect	iOS signing, TestFlight, and App Store distribution.	CREDENTIALS PENDING

Core library versions

LIBRARY	VERSION	ROLE
expo	~57.0.15	Application runtime and cross-platform toolchain
react / react-dom	19.2.3	Component model and web renderer
react-native	0.86.2	Android and iOS interface runtime
react-native-web	^0.21.2	Responsive browser rendering
@supabase/supabase-js	2.112.2	Auth, typed queries, RPCs, functions, and Realtime
AsyncStorage	2.2.0	Versioned demo persistence and client storage
expo-linking	~57.0.7	Invitation and recovery deep links
expo-splash-screen	~57.0.7	Branded native launch experience
expo-font / vector-icons	~57.0.1 / ^15.0.2	Typography and app iconography
TypeScript	~6.0.3	Static types and generated database contracts

Release environments pin Node 22.13.0. Metro 0.84.5 is forced through the React Native CLI plugin override to eliminate the previous high-severity duplicate. The workstation's unqualified PowerShell npm shim is stale, so release checks currently use the explicit npm.cmd/direct Node launchers.

SECURITY AND VERIFICATION

What has been tested and hardened

TableTime uses defense in depth: database isolation, narrow server-side workflows, controlled credentials, reproducible release checks, native runtime validation, and visible recovery paths.

Tenant isolation

RLS is enabled on all 10 app tables. Manager and employee access was verified using real authenticated identities.

Authoritative attendance

Clock and break RPCs verify auth.uid(), use server timestamps, reject invalid open states, and emit audit events.

Credential safety

Only the client-safe Supabase URL and publishable key are present in client environments. Signing and service-role secrets stay outside Git.

Production hardening

CSP, HSTS, frame denial, MIME protection, referrer controls, and least-privilege workflows are configured.

Deletion and privacy

Deletion requires reauthentication, explicit confirmation, sole-owner protection, and history-preserving de-identification.

Reliability

Retry/backoff, foreground recovery, sync status, error boundary incident IDs, and scheduled public health checks are active.

VERIFICATION	EVIDENCE
Release gate	Strict TypeScript, Expo Doctor 18/18, production web export, and zero high/critical npm findings passed on August 23.
Hosted database	Policies, grants, constraints, migrations, and rollback-safe workflow tests were checked against the cloud project.
Roles	Manager and employee production logins loaded different RLS-scoped workspaces and navigation.
Attendance	A real employee clock-in/out persisted through Supabase and reload; local demo clock, break, and clock-out were reverified.
Realtime	A second-client published shift appeared in the employee session without a page reload; the test row was removed.
Android	Replacement APK cold-launched on Android 15 with zero filtered AndroidRuntime/ReactNativeJS errors; replacement AAB signature and ZIP entries passed.
Monitoring	GitHub Production health and Release quality runs passed; public app/legal routes returned 200 and Supabase Auth gateway returned expected 401.

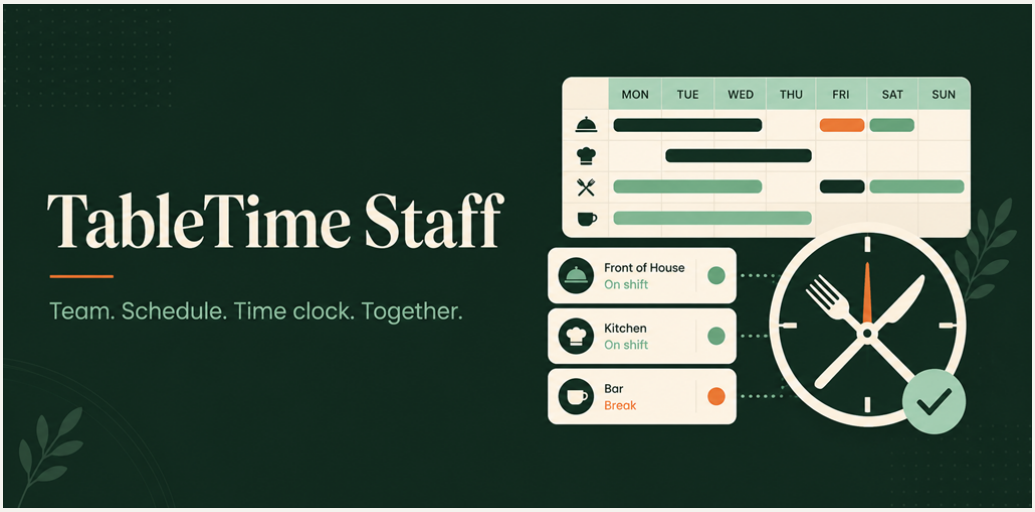
Dependency note

npm reports 11 moderate transitive Expo/Xcode build-tool advisories and zero high/critical findings. npm's forced fix would downgrade Expo and break SDK 57 compatibility, so it was intentionally rejected.

STORE READINESS

Release artifacts and listing assets

Android's safe replacement artifacts are ready for controlled testing. Brand graphics and correctly sized listing drafts now exist, while final screenshots must still be captured from signed native releases.



ASSET / ARTIFACT	VERIFIED STATE	NEXT ACTION
Android preview APK	Replacement build passed Android 15 emulator launch and visual QA.	Physical-device smoke test.
Android production AAB	VersionCode 3; SHA-256 recorded; ZIP structure and JDK signature verified.	Upload to Play internal/closed testing.
Play app icon	512 x 512 PNG with alpha; visually inspected.	Review in Play Console preview.
Play feature graphic	1024 x 500 24-bit RGB PNG without alpha; visually inspected.	Review in Play Console preview.
Google phone drafts	Five 1080 x 1920 RGB screenshots.	Replace demo UI with signed-native captures.
Apple phone drafts	Five 1080 x 2340 RGB screenshots.	Replace after iOS build and native QA.
iOS production IPA	Configuration resolves; remote build number initialized.	Privately add Apple credentials and build.

Screenshot limitation

The ten screenshot files use accepted dimensions and are appropriate for listing planning. They are not submission-ready because they were captured from the local responsive web demo and contain demo-only labels/role controls. Signed-native screenshots must replace them.

COMPLETION ROADMAP

What remains before public launch

Most remaining work is tied to private account access, truthful owner-supplied details, real-user validation, native-device evidence, and store review. These steps cannot be fabricated or safely bypassed.

ORDER	ACTION	OWNER	ESTIMATE
1	Sign into Supabase; apply production Site URL and redirect allowlist; choose whether Pro leaked-password protection is worth the expense.	Owner + Codex	1-2 h
2	Run invitation/recovery, manager request review, live breaks, employee management, owner onboarding, and disposable deletion E2E.	Codex	3-6 h
3	Install on a physical Android device; upload AAB versionCode 3 to a Play test track; capture signed-native screenshots.	Owner + Codex	2-4 h
4	Privately connect Apple credentials, build/test iOS, submit TestFlight, and capture iPhone/iPad screenshots.	Owner + Codex	3-6 h
5	Select a client-error monitoring provider, test alerting, publish owner support/legal details, and run the final release audit.	Owner + Codex	2-4 h
6	Complete store declarations, pricing/countries, reviewer access, internal testing, and production submissions.	Owner + Codex	2-5 h

Current blockers and decisions

- Supabase dashboard sign-in is needed to apply hosted Auth URLs. Leaked-password protection requires an explicit Pro-plan expense decision.
- Apple signing is blocked until the owner privately completes one paid Apple Developer credential session.
- Google Play internal/closed testing requires a developer account and app record; the AAB itself is ready.
- A real public support email, legal developer/company name, and address are required for truthful store metadata.
- The free Supabase project can sleep after low activity; choose a paid always-on plan or maintain a documented wake/health process.
- External client-error monitoring requires selection/connection of a provider; public endpoint checks are already active.

Estimated path to submission-ready

Approximately 1-3 focused working days after the required private account access and truthful owner details are available. Apple/Google review and any new-account testing requirements add separate calendar waiting time.

Key project links

Production web	https://tabletime-3qn4.vercel.app
GitHub	https://github.com/kushalsrirangam/tabletime
Privacy	https://tabletime-3qn4.vercel.app/privacy
Account deletion	https://tabletime-3qn4.vercel.app/delete-account
Replacement Android preview	https://expo.dev/accounts/kushalkings-team/projects/clockin/builds/414154eb-16bf-4230-8b51-ec936e4cd1ee
Replacement Android production	https://expo.dev/accounts/kushalkings-team/projects/clockin/builds/60d824a4-afea-4c29-b56d-3184d7e67386

Security note: this report contains no passwords, private signing keys, service-role tokens, Apple credentials, Google keys, or reviewer test passwords. Those must remain private and outside Git.